

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

Студенты гр. 4342

В.Н. Волкова,
А.Д. Киселева,
Е.С. Ковалев

Руководитель

Т.Р. Жангиров

Санкт-Петербург

2026

**ЗАДАНИЕ
НА УЧЕБНУЮ ПРАКТИКУ**

Студенты: В.Н. Волкова, А.Д. Киселева, Е.С. Ковалев

Группа: 4342

Тема практики: Генетические алгоритмы

Задание на практику:

Реализовать генетический алгоритм для поиска всех точек максимума заданного полинома.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 16.07.2026

Студенты	_____	В.Н. Волкова, А.Д. Киселева, Е.С. Ковалев
Руководитель	_____	Т.Р. Жангиров

АННОТАЦИЯ

В отчете представлена разработка генетического алгоритма для решения задачи оптимизации — поиска локальных и глобальных максимумов полинома заданной степени на определенном интервале. Описан процесс проектирования и реализации алгоритма на языке C++, а также создание графического интерфейса пользователя (GUI) с использованием фреймворка Qt. Разработанная программа включает функции настройки параметров алгоритма, поддержку ввода-вывода данных из файлов, а также пошаговую визуализацию процесса поиска экстремумов.

SUMMARY

The report presents the development of a genetic algorithm designed to solve an optimization problem: finding the local and global maxima of a polynomial of a given degree within a specified interval. It describes the algorithm's design and implementation in C++, as well as the creation of a graphical user interface (GUI) using the Qt framework. The developed program includes features for configuring algorithm parameters, support for file-based data input and output, and step-by-step visualization of the extremum-finding process.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	6
1.3. Распределение задач	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	8
2.1. Распределение ролей и структура	8
2.2. Составление списка задач	8
2.3. Создание макета GUI	9
2.4. Третья итерация.....	10
2.4.1 Polynomial.h / Polynomial.cpp	10
2.4.2 Types.h	10
2.4.3 GeneticAlgorithm.h / GeneticAlgorithm.cpp	10
2.4.4 GUI.....	12
3. Четвертая итерация	13
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	16
4 ОТЗЫВ РУКОВОДИТЕЛЯ	17
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	19
ПРИЛОЖЕНИЕ А	20
Ссылка на git-репозиторий:.....	20
main.cpp	20

ВВЕДЕНИЕ

Цель учебной практики — разработать генетический алгоритм, метод поиска оптимальных решений, который основан на принципах естественного отбора. Генетический алгоритм позволяет избежать застревания в локальных экстремумах благодаря работе с популяцией решений и использованию мутаций.

Для этого необходимо решить следующие задачи:

- Написать алгоритм на C++ для поиска всех максимумов заданного полинома.
- Разработать GUI с возможностью задавать параметры алгоритма и видеть пошаговую отрисовку графиков.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Генетические алгоритмы применяются для решения задач оптимизации, комбинаторных задач, задач, в которых нет математического решения или в которых сложная целевая функция, а также для нахождения игровых стратегий, симуляции поведения и машинного обучения.

1.2 Задачи практики

Вариант 6. Задача о поиске максимумов: для заданного полинома $f(x)$ (степень не больше 8). Необходимо с помощью генетического алгоритма найти все точки максимума (локальные и глобальные) на заданном интервале $[l, r]$.

Требование к решению:

- Программа должна иметь GUI.
- Должна быть возможность задать данные через GUI, чтение из файла, случайную генерацию.
- Алгоритмы реализуются самостоятельно без использования готовых решений.
- Настройкой параметров алгоритмов должна производиться пользователем.
- Пошаговая визуализация поиска решения (как меняется аппроксимирующая функция, текущий экстремум, текущее решение оптимизационной задачи в зависимости от популяции). Должен быть выбор переключение между разными текущими решениями.
- Возможность перейти к конечному решению пропустив отображение всех шагов.
- Должен присутствовать график изменения функции качества решения от номера популяции. График дополняется с каждым шагом алгоритма.

1.3. Распределение задач

Обязанности между членами бригады были распределены следующим образом:

Волкова В.Н. — разработка вычислительного ядра генетического алгоритма (создание структуры алгоритма GeneticAlgorithm.h, Individual, реализация основного цикла); реализация слоя-посредника (Controller) для связи алгоритма с графическим интерфейсом;

Киселева А.Д. — разработка графического интерфейса на базе фреймворка Qt; создание динамических графиков с использованием Qt Charts; отображение результатов и навигация по поколениям.

Ковалев Е.С. — написание генетического алгоритма (в том числе методов селекции, кроссовера, мутации, нишевания, функции останова), реализация вспомогательных утилит (парсинг полинома, генерация случайных параметров, работа с внешними файлами); написание отчёта по практике.

Все члены бригады принимали участие в обсуждении общей архитектуры, выборе методов и параметров алгоритма, а также в тестировании и отладке программы.

2 РЕЗУЛЬТАТЫ РАБОТЫ

В рамках первых итераций были распределены роли внутри бригады, составлен список задач, выбраны языки программирования и инструменты для работы, спроектирована первичная структура проекта и подготовлен макет графического интерфейса пользователя.

2.1. Распределение ролей и структура

Роли в бригаде были распределены следующим образом:

- Волкова В.Н. – разработка логики ГА, связь алгоритма с графическим интерфейсом;
- Киселева А.Д. – разработка GUI и графиков, работа с Qt;
- Ковалев Е.С. – разработка ГА, составление отчета.

Для разделения вычислительной логики и визуализации была спроектирована следующая архитектура проекта:

- core/ — вычислительное ядро генетического алгоритма, независимое от графического интерфейса. Включает модули вычисления функции $f(x)$ (Polynomial.h/cpp), логику селекции, скрещивания, мутации, образования ниш (GeneticAlgorithm.h/cpp), а также структуру снимка популяции для сохранения лучших особей и показателей приспособленности (GenerationSnapshot.h).
- gui/ — слой графического интерфейса пользователя. Взаимодействует только со снимками состояния популяции (snapshot), без прямого доступа к математической логике алгоритма.
- io/ — модуль ввода/вывода, отвечающий за работу с файлами, случайную генерацию и валидацию данных.

2.2. Составление списка задач

На этапе планирования был сформирован перечень задач, разделенный на два основных направления разработки.

Задачи по реализации алгоритма (C++):

1. Определить целевую функцию, структуру индивидуума и его генов, а также критерий останова алгоритма.
2. Выбрать метод отбора.
3. Реализовать несколько методов скрещивания и мутации (в зависимости от вида хромосомы) для обеспечения возможности выбора.
4. Определить тип и суть применяемых ограничений (жесткие, мягкие) и обосновать их использование.
5. Реализовать механизм образования ниш для нахождения всех максимумов функции.
6. Обеспечить возможность считывания входных данных из файла и сохранение итогового результата в выходной файл.

Задачи по реализации графической части:

1. Определить внешний вид приложения и расположение элементов управления.
2. Разработать динамически меняющиеся графики, отражающие процесс поиска решения алгоритмом в текущий момент времени.

2.3. Создание макета GUI

В ходе выполнения задач графической части был подготовлен первичный макет графического интерфейса пользователя (см. рис. 1).

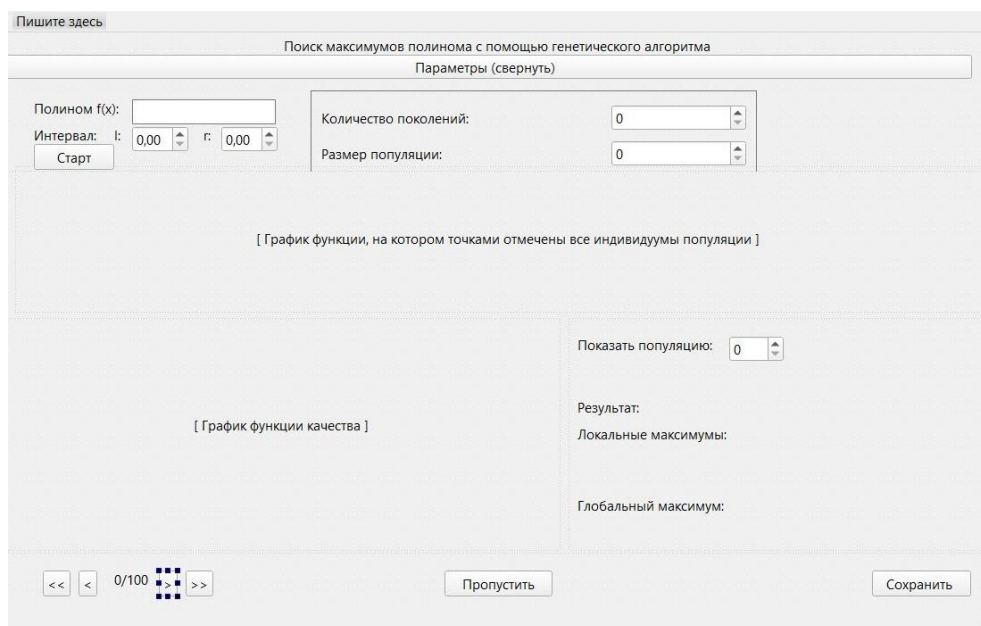


Рисунок 1 – макет GUI

2.4. Третья итерация

В рамках третьей итерации был дописан генетический алгоритм и графический пользовательский интерфейс.

2.4.1 Polynomial.h / Polynomial.cpp

Файлы содержат функцию `evaluatePolynomial()`. Функция принимает на вход вектор коэффициентов `coeffs` и значение точки `x`. Алгоритм вычисляет значения полинома.

2.4.2 Types.h

Определяет структуры данных:

`Individual`: содержит координату `x`, сырое значение приспособленности и скорректированное значение, необходимое для метода ниш.

`ProblemDefinition`: структура для хранения входных данных задачи (вектор коэффициентов полинома, границы диапазона `l` и `r`).

`GAParameters`: параметры алгоритма (размер популяции, вероятности операторов, радиус ниши). Используются перечисления `SelectionType`, `CrossoverType`, `MutationType` для выбора стратегии работы алгоритма.

`GenerationSnapshot`: структура для хранения состояния популяции на конкретной итерации, включающая средние и лучшие показатели, а также список найденных экстремумов.

2.4.3 GeneticAlgorithm.h / GeneticAlgorithm.cpp

Основной класс алгоритма.

`initializePopulation()`: Заполняет популяцию случайными значениями из диапазона `[l, r]` с использованием `std::uniform_real_distribution`. Для каждой особи рассчитывается `rawFitness`.

`step()`: Главный метод итерации. Алгоритм:

- Расчет приспособленности с учетом ниш.
- Сохранение элиты (лучшей особи).
- Селекция родителей.
- Скрещивание.

- Мутации.
- Обновление популяции и сохранение состояния.

`computeSharedFitness()`: Механизм ниш. Приспособленность каждой особи изменяется в зависимости от плотности расположения других особей в заданном радиусе.

Селекция:

- `tournamentSelect`: выбирает 2 случайные особи и возвращает лучшую из них.
- `rouletteSelect`: распределяет вероятность выбора пропорционально значению `fitness`.

Кроссовер (`arithmeticCrossover`, `blxCrossover`):

- `arithmeticCrossover`: создает потомка как среднее арифметическое родителей.
- `blxCrossover`: генерирует потомка случайно из интервала, расширенного за пределы значений родителей, что увеличивает разнообразие популяции.

Мутация:

`gaussianMutate`: добавляет к гену значение из нормального распределения.

`uniformMutate`: заменяет значение гена случайным числом в допустимом диапазоне.

`extractMaxima()`: Метод для извлечения экстремумов.

- Особи сортируются по убыванию `rawFitness`.
- Особь добавляется в список максимумов, если расстояние до ближайшего уже добавленного максимума превышает `nicheRadius`.
- Дополнительно проводится проверка сравнение значения функции в точке x со значениями в $x-0.1$ и $x+0.1$, чтобы исключить точки, находящиеся на склонах функции.

console_main.cpp: функция, позволяющая запускать алгоритм в консоли. С его помощью были проведены тестирование и отладка алгоритма.

2.4.4 GUI

По созданому графическому шаблону реализована визуальная составляющая программы. Все элементы интерфейса расположены внутри главного окна, поделенного на несколько секций (см. рис. 2):

1. Сверху располагается секция ввода данных: ввод полинома и интервала находятся в открытой части секции, при нажатии на кнопку «Параметры» разворачивается дополнительное поле для ввода параметров генетического алгоритма. В разворачиваемом поле присутствует кнопка «Загрузить», позволяющая загрузить данные из файла формата .txt, и кнопка «Сгенерировать», заполняющая параметры случайными данными.

2. Под секцией ввода данных располагается секция, содержащая график полинома с отмеченными локальными максимумами всех популяций.

3. Ниже расположена секция функции качества, отражающая зависимость качества поколения от его порядкового номера.

4. Справа от графика функции качества располагается секция отображения конкретного поколения и результатов работы алгоритма.

5. Ниже располагается секция, содержащая кнопки, позволяющие визуализировать решение пошагово, кнопку «Пропустить», предоставляющую возможность сразу получить готовое решение, и кнопку «Сохранить» для сохранения результатов.

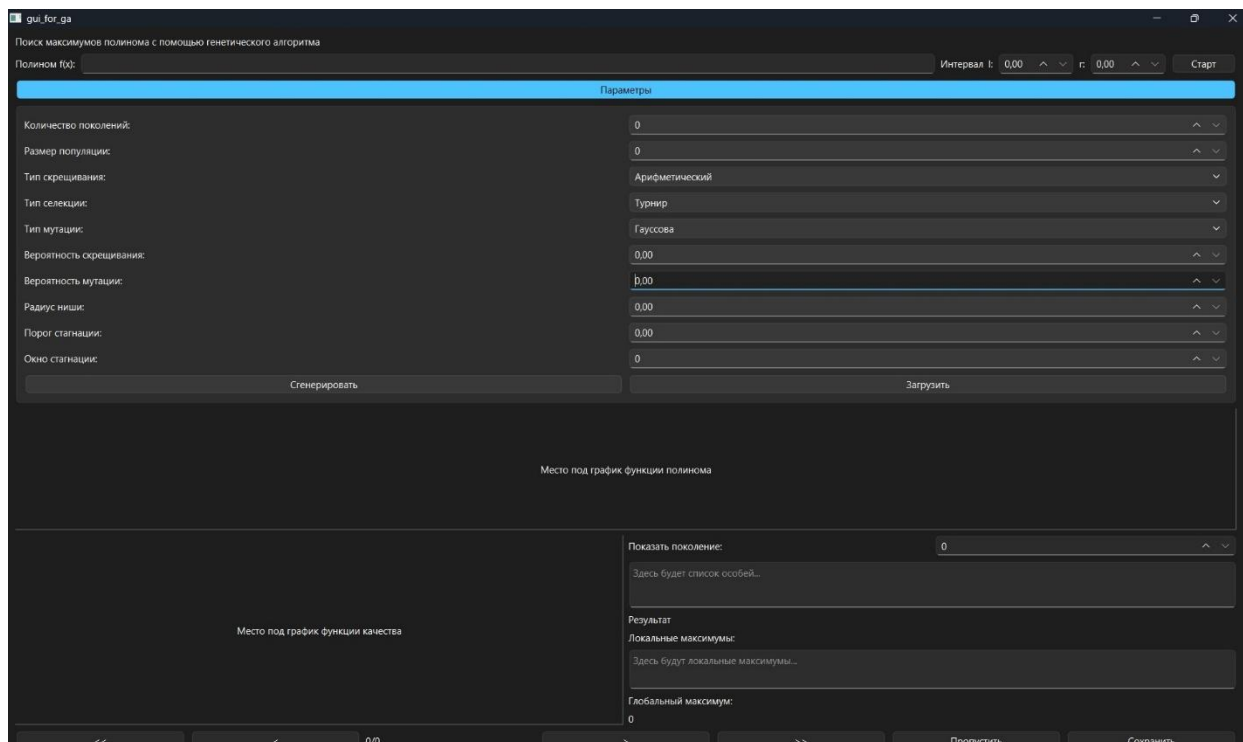


Рисунок 2 — Графический интерфейс

3. Четвертая итерация

В рамках четвертой итерации была реализована связь между графическим интерфейсом и алгоритмом, а также добавлена работа с внешними файлами.

Для обмена данными добавлены методы преобразования. Метод `inputHandle` считывает введенные пользователем значения из виджетов интерфейса, формируя из них объекты `GAParameters` и `PolynomData` для генетического алгоритма. Метод `outHandle` выполняет принимает объекты параметров и обновляет соответствующие поля на экране. Вывод списка особей текущего поколения и найденных локальных максимумов реализован в функции `updateResults`. Она добавляет числовые значения в текстовые поля интерфейса.

Визуализация данных выполнена с помощью классов модуля `Qt Charts`. Добавлен график заданного полинома на указанном интервале, построенный по вычисленным точкам с помощью `QLineSeries`. За динамическое обновление визуализации отвечает метод `updatePlots`. В зависимости от

текущего шага алгоритма он перерисовывает график функции качества и отмечает точки локальных максимумов поверх графика полинома, используя круговые маркеры (см. рис. 3).

Для разделения логики приложения был реализован отдельный класс `Controller`. Он выполняет роль посредника между главным окном `MainWindow` и вычислительной частью `GeneticAlgorithm`. Взаимодействие между модулями организовано с использованием механизма сигналов и слотов `Qt`. При взаимодействии пользователя с кнопками интерфейс генерирует сигналы, которые обрабатываются контроллером. Контроллер считывает параметры, формирует задачу, запускает алгоритм и возвращает результаты интерфейсу.

Во время работы алгоритма контроллер сохраняет историю всех поколений в виде вектора структур `GenerationSnapshot`. Эта история используется для реализации пошагового просмотра. Пользователь может перемещаться между поколениями с помощью кнопок навигации. При изменении текущего шага интерфейс автоматически обновляет списки особей, найденные максимумы и оба графика без повторных вычислений.

Дополнительно в проект добавлен функционал ввода-вывода. Реализована случайная генерация псевдослучайных, но допустимых параметров для быстрого тестирования программы. Добавлена возможность загрузки стартовых параметров из внешнего текстового файла. После завершения поиска экстремумов пользователь может нажать кнопку сохранения, чтобы выгрузить итоговые результаты работы алгоритма в отдельный файл.

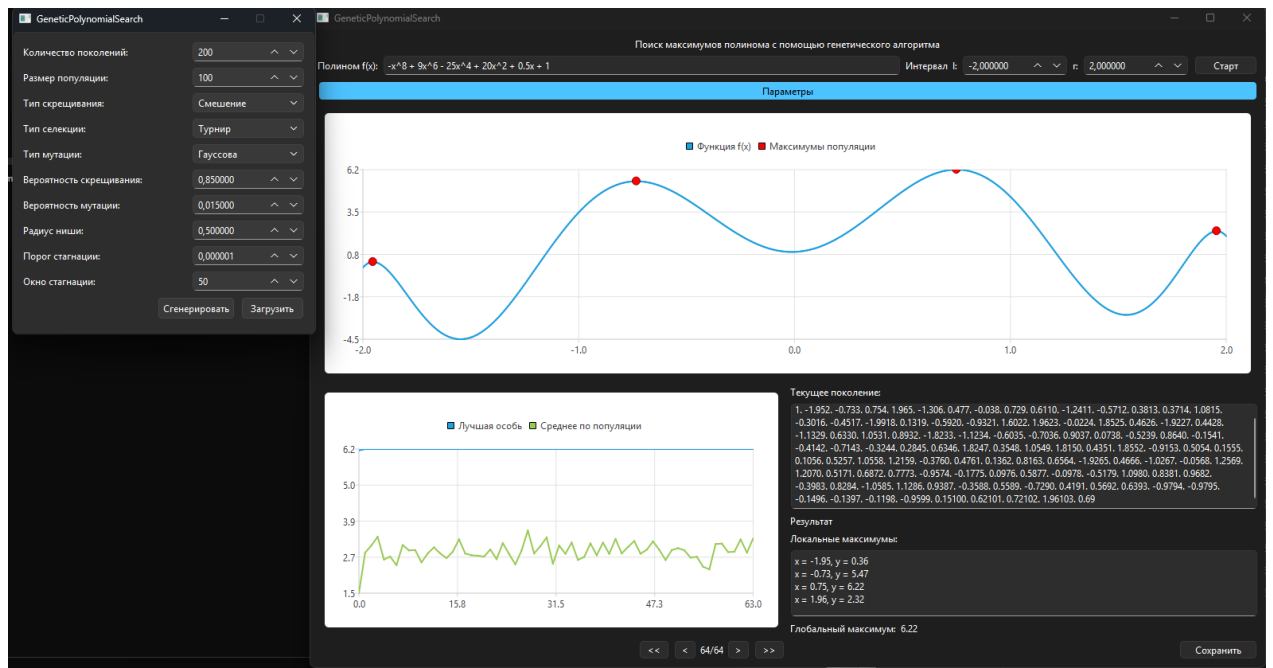


Рисунок 3 — Пример работы программы, демонстрация GUI

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В процессе работы над учебной практикой применялись следующие языки программирования, библиотеки, фреймворки и инструменты:

- Язык программирования C++ — использован для написания самого алгоритма. Выбор обусловлен высокой производительностью языка.
- Фреймворк Qt и среда разработки Qt Creator — применялись для реализации графического интерфейса пользователя. В четвертой итерации были активно использованы:
 - Механизм сигналов и слотов — для обеспечения взаимодействия между слоями контроллера и визуальными компонентами интерфейса.
 - Модуль Qt Charts — для построения интерактивных динамических графиков полинома и функции качества.
- Система контроля версий git — использовалась для организации совместной работы внутри бригады.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв

о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была достигнута основная цель — разработан генетический алгоритм для поиска оптимальных решений.

1. На языке C++ был написан алгоритм поиска максимумов заданного полинома (степенью не выше 8), включающий механизмы селекции, кроссовера, мутации и образования ниш, что позволило находить как локальные, так и глобальные максимумы функции.
2. С использованием фреймворка Qt был разработан графический интерфейс, предоставляющий возможность настраивать параметры поиска, загружать данные из файлов, сохранять результаты и осуществлять случайную генерацию условий задачи.
3. С помощью динамических графиков было реализована визуализация процесса работы алгоритма, пошаговая отрисовка поиска экстремумов и зависимости качества решения от номера популяции.

Таким образом, разработанное программное обеспечение полностью соответствует требованиям, предъявленным в техническом задании, и решает поставленные перед бригадой задачи.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 08.07.2026).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.
3. Голдберг Д. Э. Генетические алгоритмы в поиске, оптимизации и машинном обучении. — М.: Физматлит, 2003. — 304 с.
4. Страуструп Б. Язык программирования C++. Специальное издание. — М.: Бином-Пресс, 2004. — 1104 с.
5. Qt Documentation [Электронный ресурс]. URL: <https://doc.qt.io/> (дата обращения: 08.07.2026).

ПРИЛОЖЕНИЕ А

Ссылка на git-репозиторий:

https://github.com/tarackanchick/Summer_Practice.git

main.cpp

```
#include <QApplication>

#include "mainwindow.h"
#include "controller.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    MainWindow window;
    Controller controller(&window);

    window.show();

    return app.exec();
}
```